NATIONAL ECONOMICS UNIVERSITY

SCHOOL OF TECHNOLOGY

REPORT

STATISTICAL LEARNING

Topic:

CREDIT CARD FRAUD DETECTION USING XGBOOST AND DRIFT-AWARE CHUNK EVALUATION

Student: NGUYEN HUY MINH
Class: AI66A
Supervisor: Dr. Thien Van Luong

Hanoi, May 2026

PREFACE

In modern digital finance, electronic payments and online transactions provide strong convenience for users but also create a large attack surface for fraudulent activities. Credit card fraud is particularly challenging because fraudulent transactions are rare, heterogeneous, and often designed to look similar to legitimate behavior. A detection system must therefore satisfy two competing requirements: it must catch as many fraudulent transactions as possible, while avoiding excessive false alerts that inconvenience legitimate customers.

This project investigates the use of machine learning for credit card fraud detection on a large-scale tabular transaction dataset. The work focuses on exploratory data analysis, feature engineering, feature selection, class imbalance handling, XGBoost model training, threshold optimization, and drift-aware evaluation across time-sorted data chunks. The final analysis shows that strong performance on a single chunk is not sufficient: later chunks reveal both concept-like drift and threshold calibration drift, making temporal validation essential.

The report is organized as a complete machine learning project: it first defines the problem and data, then describes the preprocessing and modeling pipeline, reports experimental results, analyzes chunk-level generalization, and finally proposes a practical deployment and monitoring plan.

DECLARATION

I hereby declare that the content presented in this report is the result of my own learning, implementation, analysis, and project work. The experiments, figures, tables, and conclusions are based on the project notebook `Data_fraud.ipynb`, the generated result files, and the model evaluation outputs obtained during the project.

All external concepts and algorithms mentioned in this report, including XGBoost, Random Forest, class imbalance learning, and evaluation metrics, are cited in the References section. I take responsibility for the honesty and integrity of the analysis presented in this report.

Hanoi, May 2026
Declarant

Nguyen Huy Minh

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

ABBREVIATIONS

Abbreviation	Full Form
AI	Artificial Intelligence
ML	Machine Learning
EDA	Exploratory Data Analysis
RF	Random Forest
XGBoost	Extreme Gradient Boosting
PR-AUC	Area Under the Precision–Recall Curve
ROC-AUC	Area Under the Receiver Operating Characteristic Curve
TP	True Positive
FP	False Positive
TN	True Negative
FN	False Negative
F1	Harmonic mean of precision and recall
MI	Mutual Information
PSI	Population Stability Index

ABSTRACT

This report presents a credit card fraud detection project using tree-based machine learning models, with XGBoost as the final modeling approach. The dataset is highly imbalanced: in the first time-sorted chunk used for detailed exploratory analysis, fraudulent transactions account for only 1.0597% of all transactions. Because of this imbalance, accuracy is not a reliable evaluation metric; instead, the project uses PR-AUC, precision, recall, F1-score, and confusion matrix analysis.

Exploratory data analysis shows that transaction amount and category are the strongest fraud signals, while distance between customer and merchant and time-of-day have weak standalone predictive power. Feature selection reduces the model-ready feature set from 41 features to 25 features with almost no loss in PR-AUC. XGBoost models are trained under different imbalance-handling strategies. On data chunk 01, both tuned XGBoost candidates achieve strong test performance, with PR-AUC above 0.96 and F1-score above 0.95.

However, evaluating later chunks reveals that strong chunk-01 performance does not guarantee stable generalization. Some chunks show true ranking degradation, while others show low F1-score despite high PR-AUC, indicating threshold calibration drift rather than concept drift. The final recommendation is to use `xgb_class_weight` as the production baseline, combined with dynamic thresholding, walk-forward validation, and drift monitoring.

CHAPTER 1. PROBLEM DEFINITION

1.1 Rationale of the Project

Credit card fraud detection is a critical application of machine learning in financial security. In a real transaction stream, fraudulent behavior is rare but costly. A trivial classifier that predicts every transaction as legitimate can achieve very high accuracy, yet it fails completely from a risk-management perspective. Therefore, fraud detection must be evaluated using metrics that focus on the minority class and reflect operational trade-offs.

The project aims to build a model that can identify fraudulent transactions using transaction attributes, customer behavior features, merchant-related features, and time-based information. The key challenge is to balance fraud recall with false alert control. Missing fraud cases creates financial loss, while over-flagging legitimate transactions damages customer experience and increases operational cost.

1.2 Problem Statement

Let each transaction be represented by a feature vector $\mathbf{x}_i$ and binary target $y_i \in \{0,1\}$, where $y_i = 1$ indicates fraud. The objective is to learn a model $f(\mathbf{x}_i)$ that outputs a fraud probability:

$$\hat{p}_i = f(\mathbf{x}_i) = P(y_i = 1 \mid \mathbf{x}_i). \tag{1}$$

A decision threshold τ is then applied:

$$\hat{y}_i = \begin{cases} 1, & \hat{p}_i \geq \tau, \\ 0, & \hat{p}_i < \tau. \end{cases} \tag{2}$$

The model must perform well not only on the first training chunk but also across future time-sorted chunks. This makes temporal validation and drift analysis important parts of the project.

1.3 Why Fraud Detection Is Hard

The main difficulties observed in this project are:

- **Severe class imbalance**: fraud is only about 1% of transactions.
- **Non-linear feature interactions**: category, amount, job, and customer behavior

interact in non-linear ways.

- **Potential dataset artifacts**: some categories appear nearly deterministic in chunk 01.
- **Temporal instability**: later chunks show different behavior from chunk 01.
- **Threshold sensitivity**: F1-score can collapse if the validation threshold is no longer suitable for future data.

CHAPTER 2. DATA OVERVIEW

2.1 Data Source and Chunking

The project uses a large credit card transaction dataset stored in parquet chunks. The full processed dataset contains 16 time-sorted chunks. Each chunk contains approximately 536K transactions. The detailed EDA and initial model tuning are performed on chunk 01, while chunks 02–16 are used for cross-chunk validation and drift analysis.

Table 1: Overall dataset summary

Attribute	Value
Total transactions	8,580,255
Fraud transactions	94,806
Overall fraud rate	1.105%
Number of time-sorted chunks	16
Rows per chunk	$\approx$ 536K
Evaluated future chunks	02–16

2.2 Chunk 01 Summary

Chunk 01 contains 536,266 transactions and 44 raw columns. It has no missing values across all columns. Important column groups include identifiers, time information, transaction amount, customer profile attributes, merchant attributes, rolling customer behavior, rolling merchant behavior, and the binary target variable `is_fraud`.

Table 2: Chunk 01 data quality summary

Attribute	Value
Rows	536,266
Columns	44
Missing values	0
Unique transaction IDs	536,266
Unique accounts/cards	8,757
Unique merchants	655
Unique jobs	639
Unique categories	14
Transaction period	2022-05-06 to 2022-05-30

2.3 Target Variable

The target variable is `is_fraud`. In chunk 01, fraud accounts for only 1.0597% of transactions.

Table 3: Target distribution in chunk 01

Class	Count	Rate
Non-fraud	530,583	98.9403%
Fraud	5,683	1.0597%

Figure 1: Target distribution in chunk 01

This confirms that accuracy should not be used as the main metric. A classifier predicting all transactions as non-fraud would still achieve nearly 99% accuracy, while failing to detect all fraud cases.

CHAPTER 3. EXPLORATORY DATA ANALYSIS

3.1 Time-Based Analysis

The transaction period of chunk 01 covers May 2022. The age range of customers is 14–95, with a median age of 41. Fraud rate by hour does not show a strong concentration at night, which contradicts a common heuristic that fraudulent transactions are mostly nocturnal.

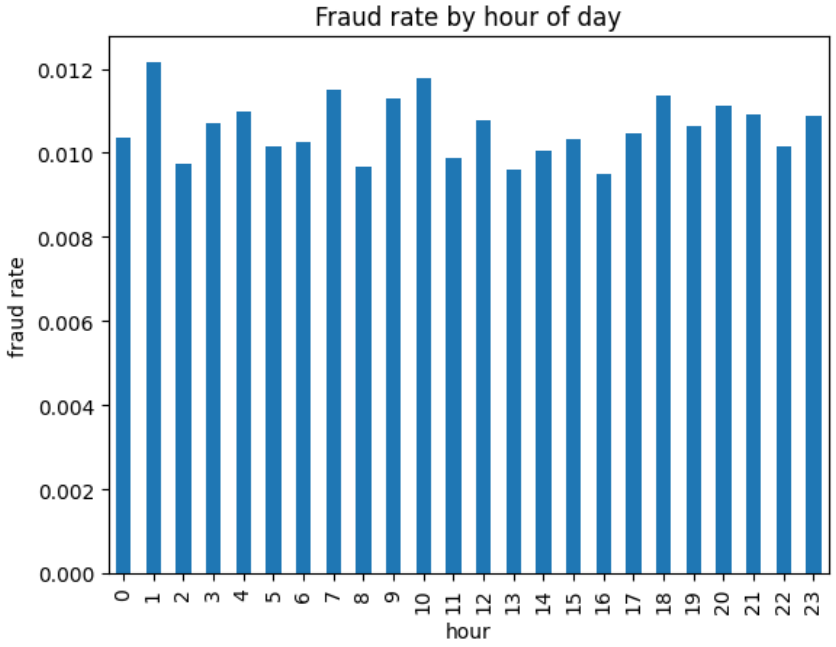
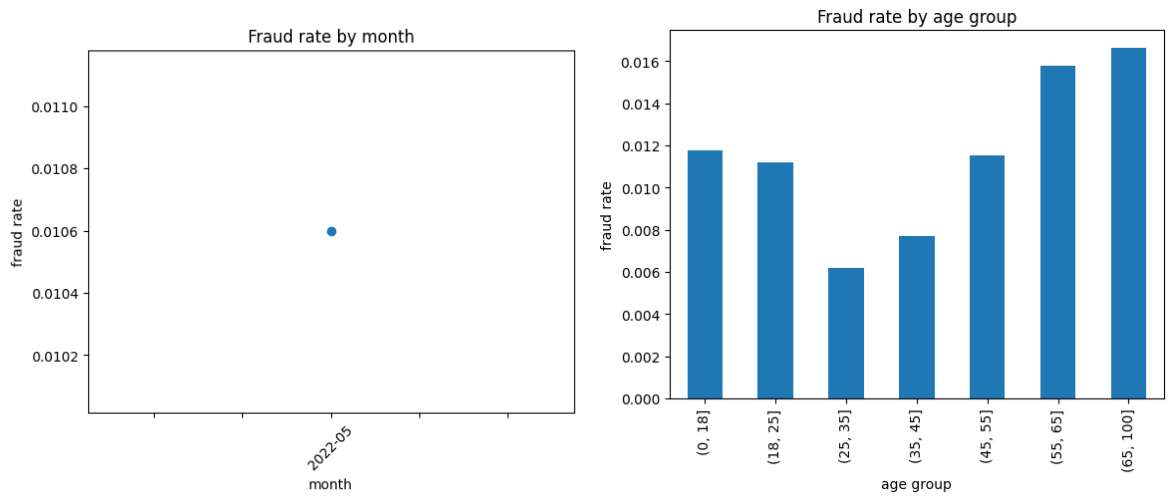


Figure 2: Fraud rate by transaction hour

Weekend behavior is more informative. In chunk 01, weekday transactions have a higher fraud rate than weekend transactions.

Table 4: Fraud rate by weekday/weekend in chunk 01

Segment	Total	Fraud	Fraud rate
Weekday	285,937	3,778	1.3213%
Weekend	250,329	1,905	0.7610%



(a) Fraud rate by month

(b) Fraud rate by age bin

Figure 3: Time and age-related fraud patterns

The age-bin analysis suggests that older groups have higher fraud rates in chunk 01, particularly the 55–65 and 65–100 age ranges. This motivates using `age_bin_ord` rather than only raw age.

3.2 Categorical Analysis

Categorical columns have different cardinalities. Category and gender are low-cardinality variables, while merchant and job are high-cardinality variables.

Table 5: Categorical column summary

Column	Unique values	Null count	Encoding choice
gender	2	0	Binary encoding
category	14	0	One-hot encoding
merchant	655	0	Frequency encoding
job	639	0	Frequency encoding

Category is one of the strongest signals in chunk 01. Several categories appear to have a fraud rate of 100%, which is useful for prediction but must be treated carefully because it may reflect a dataset artifact.

Table 6: Selected category fraud rates in chunk 01

Category	Total	Fraud	Fraud rate
grocery_pos	1,394	1,394	100.0%
shopping_net	1,299	1,299	100.0%
misc_net	744	744	100.0%
shopping_pos	659	659	100.0%
gas_transport	458,338	465	0.1015%
grocery_net	72,804	94	0.1291%

Gender has only a small difference: male transactions have a fraud rate of 1.112%, while female transactions have a fraud rate of 1.008%. Job is more informative than gender and merchant identity. Some job groups show fraud rates several times larger than the global baseline.

Table 7: Top job fraud rates in chunk 01

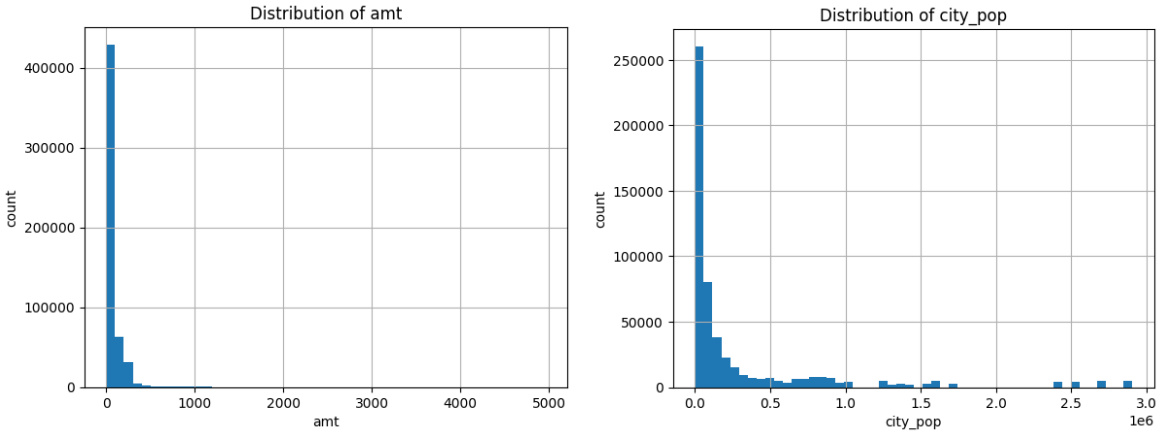
Job	Fraud rate
Scientist, product/process development	9.82%
Systems developer	7.00%
Theatre manager	6.38%
Copywriter, advertising	5.99%
Holiday representative	5.42%

3.3 Numeric Analysis

Transaction amount is the strongest numeric feature. Fraudulent transactions are much larger on average than legitimate transactions.

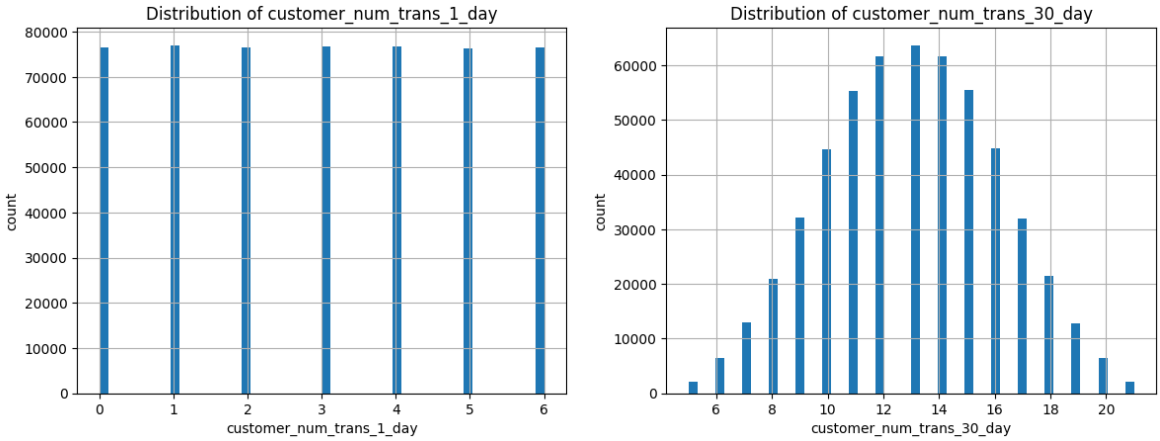
Table 8: Selected numeric comparison between non-fraud and fraud

Feature	Non-fraud mean	Fraud mean	Non-fraud median	Fraud m
amt	78.57	534.61	61.51	
trans_time_hrs	11.50	11.48	12.00	
trans_date_is_weekend	0.468	0.335	0.00	
merchant_risk_30_day	16.12	16.25	17.00	
customer_num_trans_30_day	13.00	13.04	13.00	



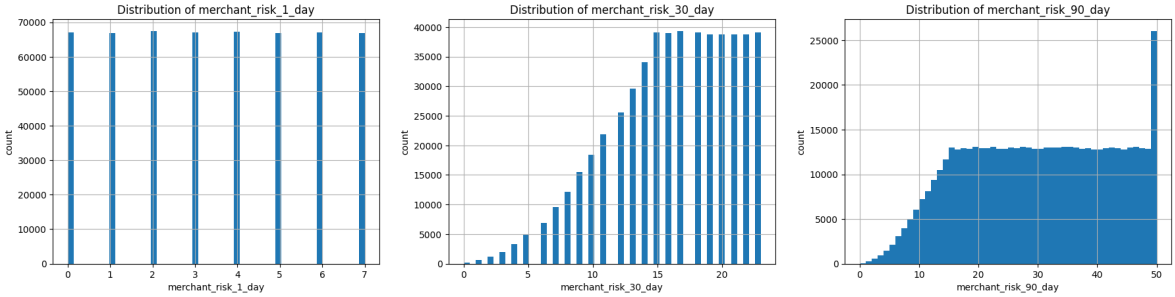
(a) `amt` **(b)** `city_pop`

Figure 4: Numeric distributions I



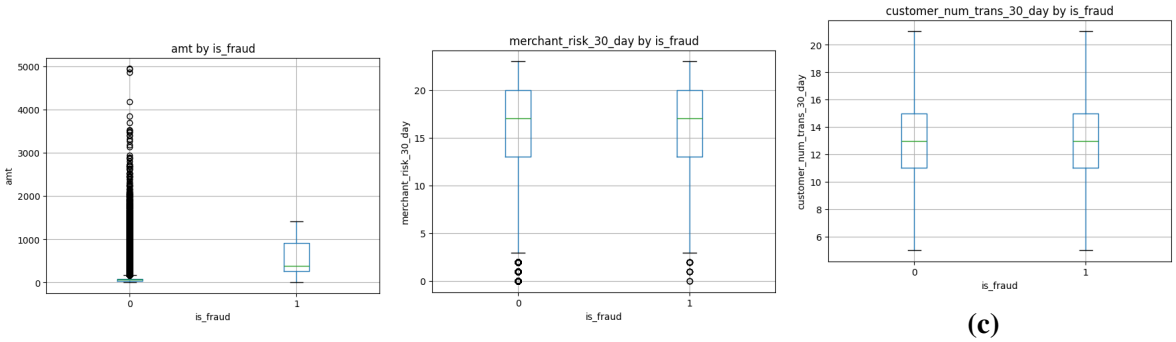
(a) `customer_num_trans_1_day` **(b)** `customer_num_trans_30_day`

Figure 5: Numeric distributions II



(a) `merchant_risk_1_day` **(b)** `merchant_risk_30_day` **(c)** `merchant_risk_90_day`

Figure 6: Merchant risk distributions



(a) `amt` **(b)** `merchant_risk_30_day` **(c)** `customer_num_trans_30_day`

Figure 7: Boxplots by target

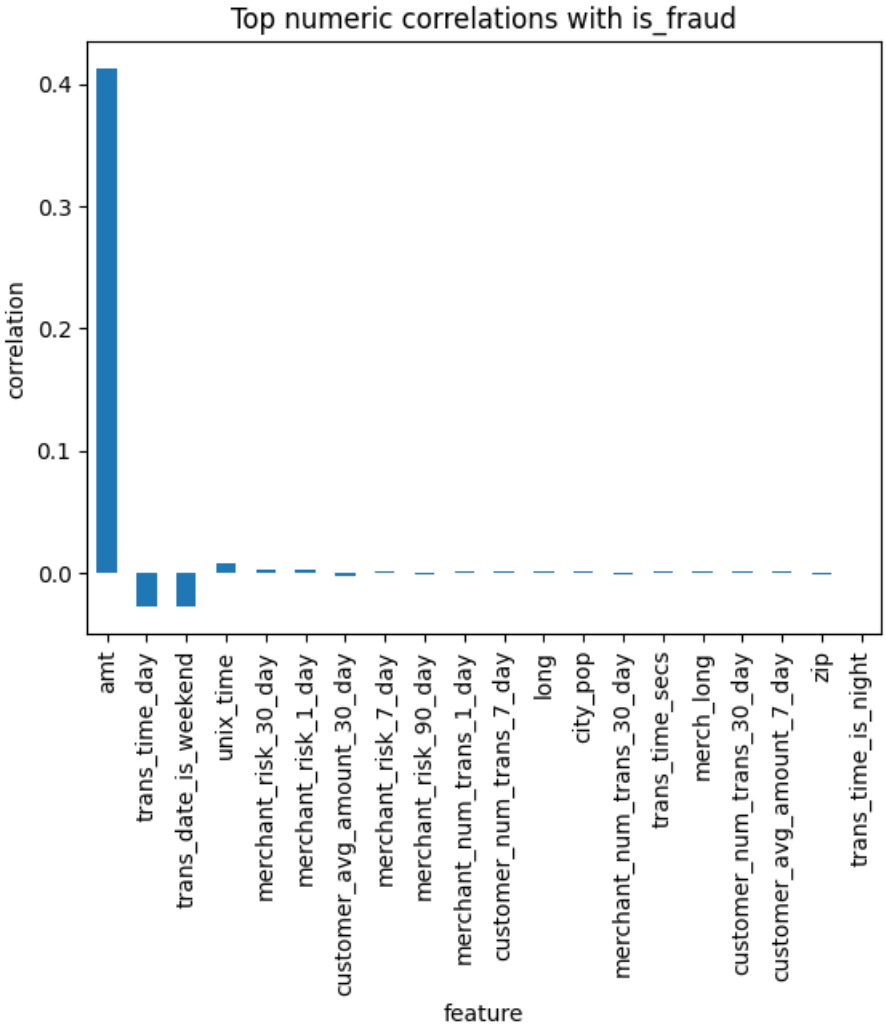


Figure 8: Top numeric correlations with fraud label

3.4 Geographic Distance Analysis

The distance between the customer and merchant is computed using the Haversine formula. The distribution is almost identical for fraud and non-fraud classes.

Table 9: Customer–merchant distance by class

Class	Mean km	Median km	Std km
Non-fraud	76.55	78.73	29.21
Fraud	76.77	78.60	29.34

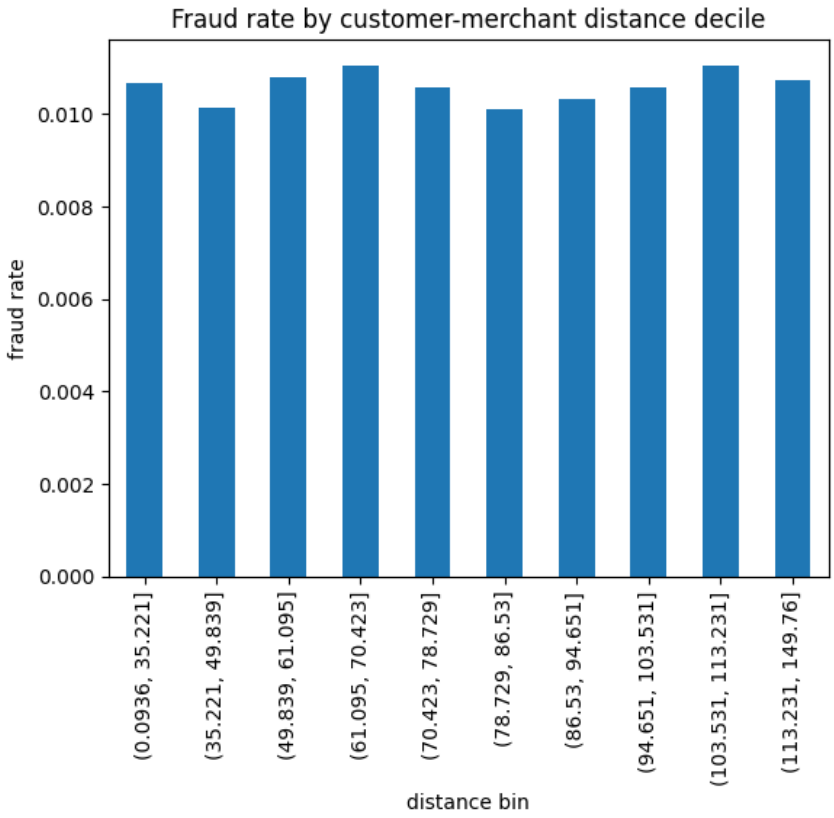


Figure 9: Fraud rate by distance decile

The fraud rate is nearly flat across distance bins. Therefore, `customer_merchant_distance_km`, `merch_lat`, and `merch_long` are not retained in the final feature set.

CHAPTER 4. METHODOLOGY

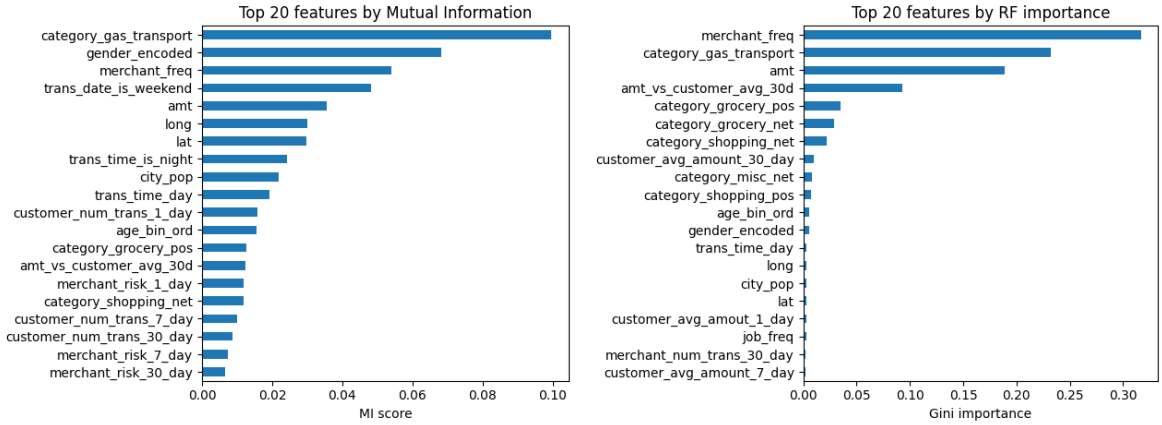
4.1 Feature Engineering

The feature engineering process converts raw transaction data into model-ready numerical features. Gender is encoded as a binary variable. Category is one-hot encoded due to its low cardinality. Merchant and job are frequency-encoded because their cardinalities are high. Age is converted into ordinal age bins. Additional ratio features are created, including the current amount relative to the customer's 30-day average amount.

The final transformed dataset contains no remaining object columns and has 43 columns before train/test splitting. After sorting by `unix_time`, chunk 01 is split into 429,012 training rows and 107,254 test rows.

4.2 Feature Selection

Feature selection uses two complementary approaches: Mutual Information and RandomForest feature importance. A feature is kept if it appears in the top-20 of either ranking, after removing highly collinear drop candidates.



(a) Mutual Information **(b)** RandomForest importance

Figure 10: Feature ranking plots

Table 10: Feature selection impact

Feature set	Number of features	PR-AUC
All model-ready features	41	0.9588
Selected features	25	0.9576
Difference	-16	-0.0013

The selected feature set reduces dimensionality by 39% while losing only 0.0013 PR-AUC. This indicates that many features are redundant or weak, and a smaller feature set can improve robustness and reduce training cost.

4.3 Handling Imbalanced Data

Two imbalance-handling strategies are compared for XGBoost:

- **Class weighting**: keep the original data distribution and use `scale_pos_weight`.
- **Undersampling**: reduce the majority class to a 10:1 negative-to-positive ratio.

Class weighting is preferred for the final recommendation because it preserves the original distribution and is more stable across future chunks.

4.4 Modeling Approach

The notebook evaluates RandomForest and XGBoost variants with baseline, class-weighted, and undersampled strategies. XGBoost provides the strongest validation PR-AUC and is selected for hyperparameter tuning.

Table 11: Initial model and imbalance-strategy comparison on chunk 01

Name	Strategy	Train size	Validation PR-AUC
rf_baseline	baseline	343,209	0.9417
rf_class_weight	class weight	343,209	0.9444
rf_undersample	undersample	39,886	0.9587
xgb_baseline	baseline	343,209	0.9655
xgb_class_weight	class weight	343,209	0.9655
xgb_undersample	undersample	39,886	0.9724

CHAPTER 5. EXPERIMENTS, EVALUATION, AND RESULTS

5.1 Evaluation Strategy

The project uses PR-AUC for model ranking and F1-score for threshold-based classification quality. Accuracy is excluded from the main analysis because the dataset is severely imbalanced. For threshold tuning, the validation Precision–Recall curve is used to select the threshold that maximizes F1:

$$\tau^* = \arg\max_{\tau} F1(y_{\text{val}}, \hat{p}_{\text{val}} \geq \tau). \tag{3}$$

The selected threshold is then applied to the test set.

5.2 Hyperparameter Tuning

The top two XGBoost candidates are tuned with Optuna using validation PR-AUC as the objective.

Table 12: Best tuned XGBoost hyperparameters

Model	n_est	depth	lr	subsample	colsample	child	alpha	lambda	gamma
xgb_undersample	150	8	0.016664	0.948150	0.781684	6	0.189588	0.007976	0.660841
xgb_class_weight	400	6	0.015384	0.880095	0.754347	1	0.155126	0.005929	3.799895

5.3 Chunk 01 Final Evaluation

Both tuned XGBoost models perform strongly on chunk 01 test data.

Table 13: Final tuned model results on chunk 01 test set

Model	PR-AUC	Threshold	F1	TN	FP	FN	TP
xgb_undersample	0.9614	0.7484	0.9564	106,356	0	75	823
xgb_class_weight	0.9623	0.9722	0.9564	106,356	0	75	823

For the fraud class, both final models reach precision 1.0000 and recall 0.9165 on chunk 01 test data. This means that, in this test segment, the models do not produce false positive alerts at the selected thresholds, while still detecting 823 out of 898 fraud cases.

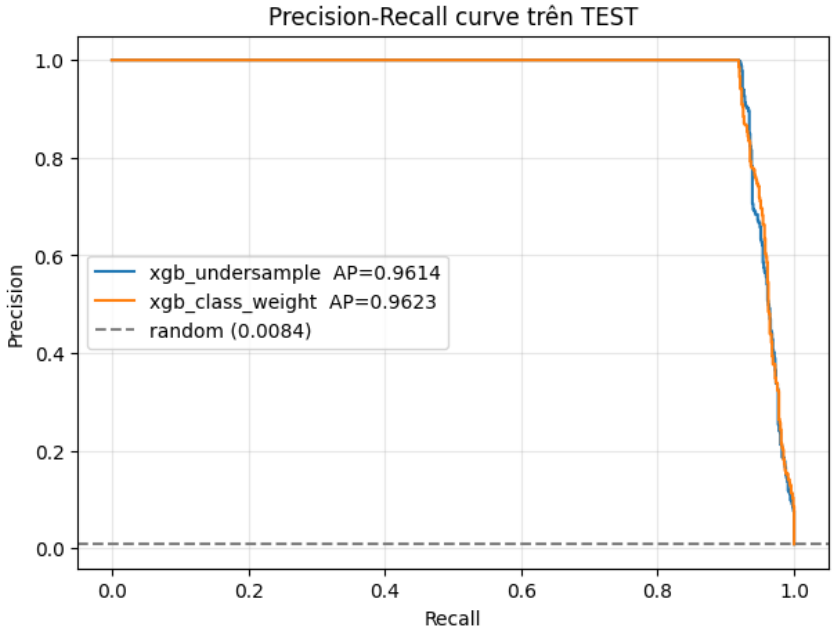


Figure 11: Precision–Recall curve of tuned XGBoost models on chunk 01

5.4 Cross-Chunk Evaluation

The chunk 01 result is strong, but a production model must generalize to later time periods. Therefore, the tuned parameters are applied to chunks 02–16. Chunk 16 is excluded from quality conclusions because its test segment contains no fraud cases.

Table 14: Mean cross-chunk performance excluding no-positive test chunks

Model	Mean test PR-AUC	Mean F1 at validation threshold	Best diagnostic F1
xgb_class_weight	0.8025	0.6031	0.7935
xgb_undersample	0.7736	0.5957	0.7814

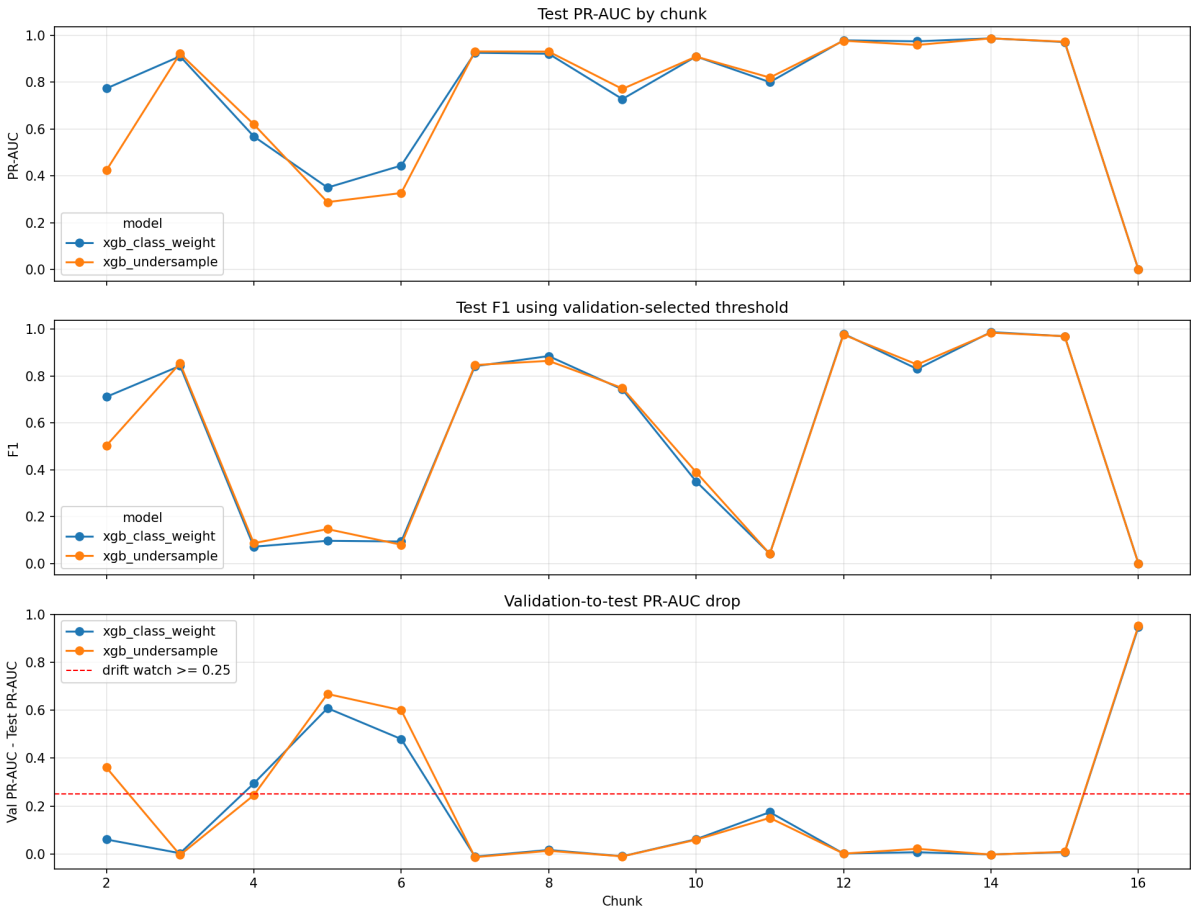


Figure 12: Chunk-level PR-AUC, F1, and validation-to-test PR-AUC drop

The cross-chunk results show that strong performance on chunk 01 does not guarantee stable performance on future chunks. The average PR-AUC decreases substantially when evaluating across later chunks.

CHAPTER 6. DRIFT ANALYSIS

6.1 Separating Concept Drift and Threshold Drift

Low F1-score alone is not sufficient evidence of concept drift. The report separates two possible failure modes:

- **Ranking/concept drift**: PR-AUC drops strongly, meaning the model no longer ranks fraud above non-fraud reliably.

- **Threshold drift**: PR-AUC remains high, but F1-score becomes low because the validation threshold no longer matches the test score distribution.

Table 15: Diagnosis of selected low-F1 chunks using `xgb_class_weight`

Chunk	Val PR-AUC	Test PR-AUC	F1 at val threshold	Diagnosis
2	0.8338	0.7726	0.7109	OK
4	0.8629	0.5682	0.0715	Ranking drift suspected
5	0.9586	0.3500	0.0966	Concept/label drift likely
6	0.9238	0.4439	0.0935	Concept/label drift likely
9	0.7184	0.7274	0.7427	OK
10	0.9712	0.9091	0.3500	Threshold calibration drift
11	0.9745	0.7997	0.0414	Threshold calibration drift

6.2 Concept or Label Drift Candidates

Chunks 5 and 6 are the strongest concept or label drift candidates. Their validation PR-AUC values are high, but their test PR-AUC values collapse to 0.3500 and 0.4439 respectively. Even the best diagnostic test F1 remains low. This suggests that the model is not only miscalibrated; it is no longer ranking fraud reliably in those test segments.

6.3 Threshold Calibration Drift

Chunks 10 and 11 behave differently. Their test PR-AUC values remain relatively high, but their F1-scores at validation-selected thresholds are very low. The best diagnostic F1 values are much higher, indicating that the model still ranks fraud well but the threshold is no longer appropriate.

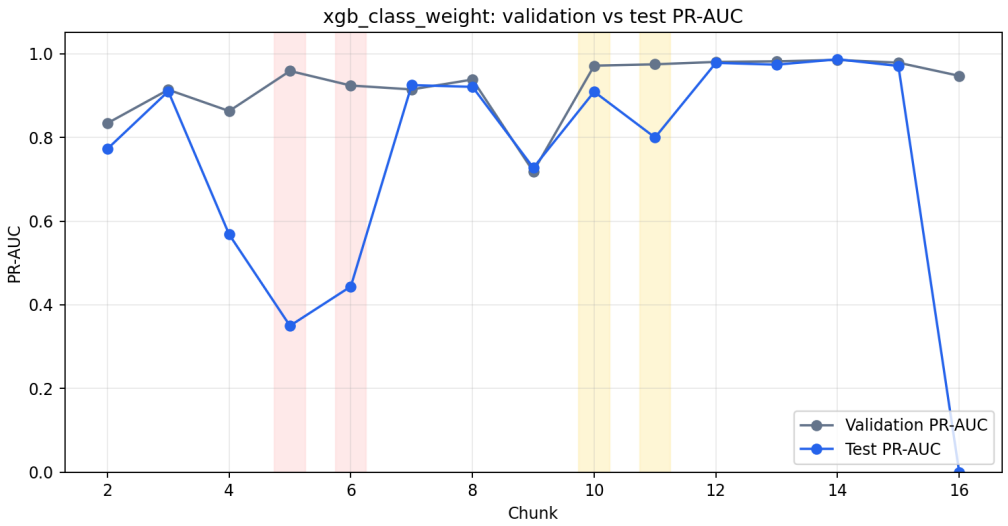


Figure 13: Validation vs test PR-AUC for `xgb_class_weight`

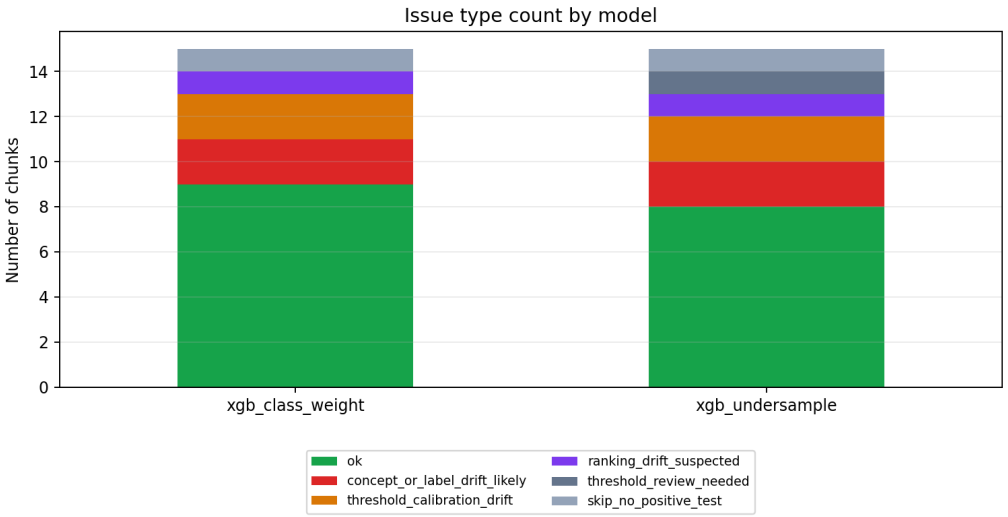


Figure 14: Issue type counts across chunks 02–16

This distinction is operationally important. If a chunk suffers from threshold drift, the system should recalibrate the threshold before retraining the entire model. If a chunk suffers from ranking or concept drift, retraining with a more recent rolling window becomes more appropriate.

CHAPTER 7. DEPLOYMENT PLAN

7.1 Recommended Pipeline

Based on the full analysis, the recommended pipeline is:

1. Use the selected 25-feature set.
2. Use XGBoost with class weighting as the production baseline.
3. Use time-based walk-forward validation.
4. Select thresholds dynamically using the most recent validation window.
5. Monitor PR-AUC, F1-score, fraud rate, and feature distribution drift.
6. Retrain only when ranking quality degrades, not merely when F1-score drops.

7.2 Inference Contract

The inference pipeline should require the same feature order used during training. Raw inputs must be transformed into the selected model features, including amount ratios, age bins, category one-hot features, frequency encodings, rolling customer behavior, and merchant risk features. The model should output both a fraud probability and a binary decision based on the current operating threshold.

7.3 Monitoring

Monitoring should distinguish between model ranking degradation and threshold instability. Recommended monitoring indicators include:

- PR-AUC and F1-score over time.
- Fraud rate by time window.
- False positive and false negative counts.
- PSI or KS statistics for important features such as `amt`, category indicators, `merchant_freq`, `job_freq`, and rolling behavior features.
- Difference between validation PR-AUC and future test PR-AUC.

CHAPTER 8. CONCLUSION AND FUTURE WORK

8.1 Executive Summary

The project shows that XGBoost is highly effective for credit card fraud detection on the analyzed tabular dataset. Chunk 01 results are very strong, with PR-AUC above 0.96 and F1-score above 0.95. EDA shows that amount and category are the strongest signals, while distance and time-of-day are weak standalone signals. Feature selection reduces the number of model features from 41 to 25 with almost no PR-AUC loss.

However, cross-chunk evaluation reveals that the model is not uniformly stable over time. Chunks 5 and 6 show evidence of concept or label drift, while chunks 10 and 11 mainly show threshold calibration drift. Therefore, a realistic fraud detection system should not rely on a single static threshold or a single validation split.

The final recommendation is to use `xgb_class_weight` with the selected feature set, dynamic thresholding, walk-forward validation, and drift-aware monitoring.

8.2 Limitations

Several limitations remain:

- Some category patterns appear nearly deterministic in chunk 01 and may reflect dataset artifacts.
- Frequency encoding was computed within the analysis pipeline and should be adapted carefully for production.
- The dataset is synthetic or simulation-based, so real production data may contain additional noise.
- The report does not implement an online learning system; it only proposes a monitoring and retraining strategy.

8.3 Future Work

Future work should focus on:

- Implementing walk-forward training using rolling windows.
- Adding probability calibration methods such as Platt scaling or isotonic calibration.
- Testing cost-based threshold optimization instead of F1-only thresholding.

- Building a production-ready inference API.
- Extending drift detection with PSI, KS-test, and label-delay-aware monitoring.

REFERENCES

[1] T. Chen and C. Guestrin. *XGBoost: A Scalable Tree Boosting System*. Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 2016.

[2] L. Breiman. *Random Forests*. Machine Learning, 45(1), 5–32, 2001.

[3] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer. *SMOTE: Synthetic Minority Over-sampling Technique*. Journal of Artificial Intelligence Research, 16, 321–357, 2002.

[4] F. Pedregosa et al. *Scikit-learn: Machine Learning in Python*. Journal of Machine Learning Research, 12, 2825–2830, 2011.

[5] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama. *Optuna: A Next-generation Hyperparameter Optimization Framework*. Proceedings of the 25th ACM SIGKDD International Conference, 2019.